



**POLITÉCNICO
ESTELLA**

ASIGNATURA: ACCESO A DATOS

**GRADO SUPERIOR EN DESARROLLO DE
APLICACIONES MULTIPLATAFORMA**

AD TAREA 7

Presentado por: Eugen Moga

Fecha: 18/03/2026

Índice

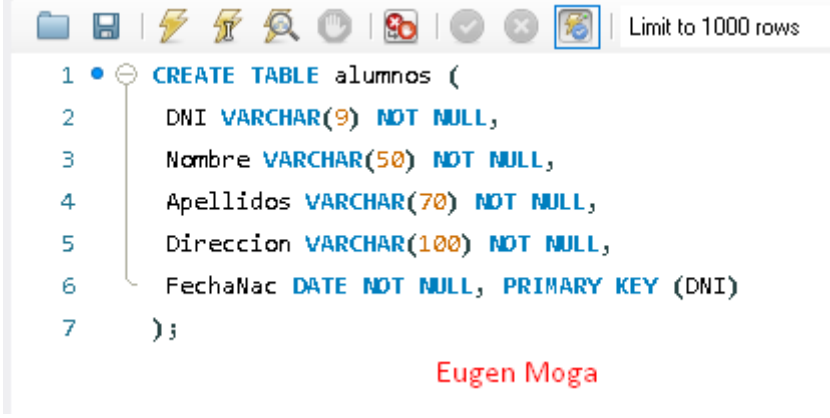
Creación de SQL 1

Creación de componente 3

Proyecto AD07_Prueba..... 14

Creación de SQL

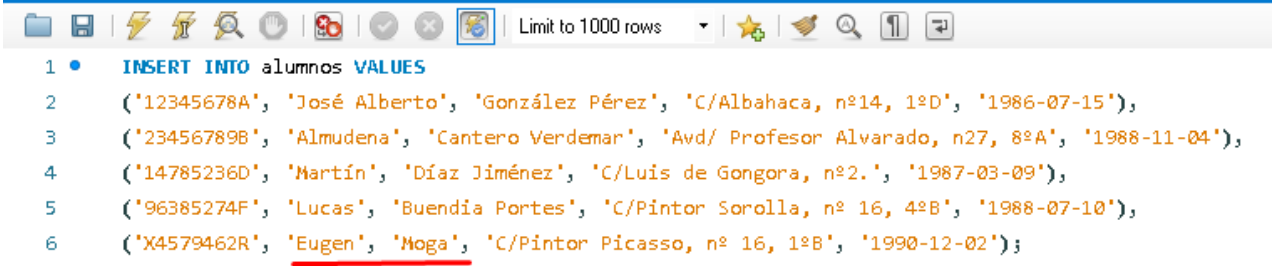
He aprovechado la base de datos RRHH de la tarea del tema 3 donde he agregado la tabla alumnos y matriculas



```
1 CREATE TABLE alumnos (  
2     DNI VARCHAR(9) NOT NULL,  
3     Nombre VARCHAR(50) NOT NULL,  
4     Apellidos VARCHAR(70) NOT NULL,  
5     Direccion VARCHAR(100) NOT NULL,  
6     FechaNac DATE NOT NULL, PRIMARY KEY (DNI)  
7 );
```

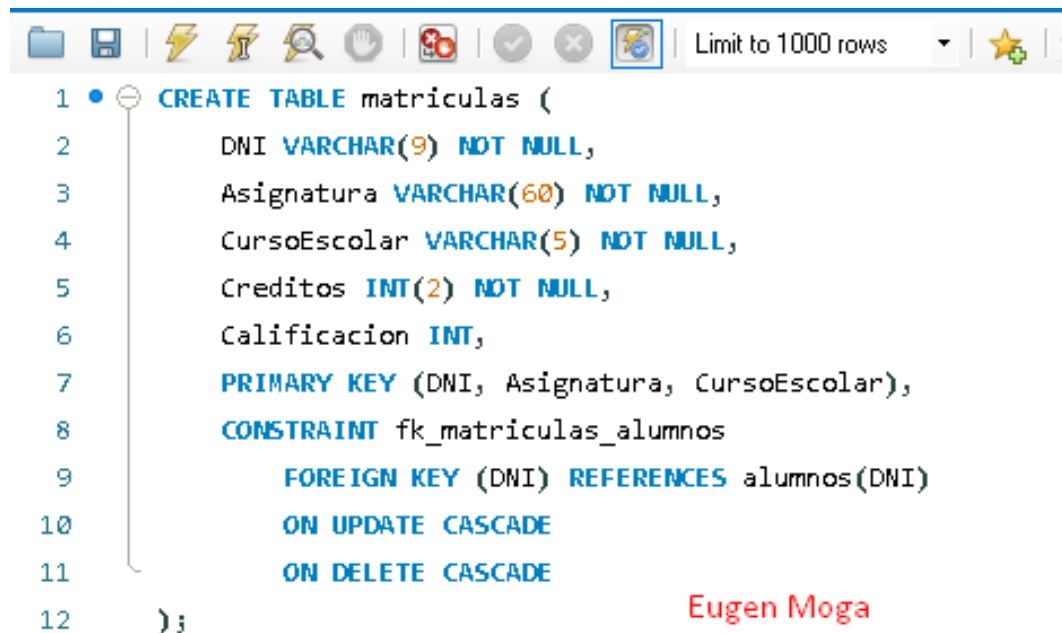
Eugen Moga

Y he agregado unos cuantos alumnos.



```
1 INSERT INTO alumnos VALUES  
2 ('12345678A', 'José Alberto', 'González Pérez', 'C/Albahaca, nº14, 1ºD', '1986-07-15'),  
3 ('23456789B', 'Almudena', 'Cantero Verdeman', 'Avd/ Profesor Alvarado, nº27, 8ºA', '1988-11-04'),  
4 ('14785236D', 'Martín', 'Díaz Jiménez', 'C/Luis de Gongora, nº2.', '1987-03-09'),  
5 ('96385274F', 'Lucas', 'Buendia Portes', 'C/Pintor Sorolla, nº 16, 4ºB', '1988-07-10'),  
6 ('X4579462R', 'Eugen', 'Moga', 'C/Pintor Picasso, nº 16, 1ºB', '1990-12-02');
```

He creado la tabla matriculas con DNI, Asignatura y CursoEscolar como clave compuesta ya que un alumno puede estar matriculado en el mismo modulo en diferentes cursos escolares si suspende el módulo.



```
1 • CREATE TABLE matriculas (  
2     DNI VARCHAR(9) NOT NULL,  
3     Asignatura VARCHAR(60) NOT NULL,  
4     CursoEscolar VARCHAR(5) NOT NULL,  
5     Creditos INT(2) NOT NULL,  
6     Calificacion INT,  
7     PRIMARY KEY (DNI, Asignatura, CursoEscolar),  
8     CONSTRAINT fk_matriculas_alumnos  
9         FOREIGN KEY (DNI) REFERENCES alumnos(DNI)  
10        ON UPDATE CASCADE  
11        ON DELETE CASCADE  
12 );
```

Eugen Moga

Agrego datos en la tabla matriculas

```
1  -- Datos de matriculas  
2 • INSERT INTO matriculas (DNI, Asignatura, CursoEscolar, Creditos, Calificacion) VALUES  
3  ('12345678A', 'Programación', '23/24', 8, 7),  
4  ('12345678A', 'Bases de Datos', '23/24', 6, 8),  
5  ('12345678A', 'Lenguajes de Marcas', '23/24', 4, 9),  
6  
7  ('23456789B', 'Programación', '24/25', 8, 6),  
8  ('23456789B', 'Sistemas Informáticos', '25/26', 6, 7),  
9  ('23456789B', 'Entornos de Desarrollo', '24/25', 3, 8),  
10  
11  ('14785236D', 'Bases de Datos', '23/24', 6, 5),  
12  ('14785236D', 'Programación', '23/24', 8, 6),  
13  
14  ('96385274F', 'Lenguajes de Marcas', '23/24', 4, 10),  
15  ('96385274F', 'Entornos de Desarrollo', '23/24', 3, 9),  
16  
17  ('X4579462R', 'Programación', '25/26', 8, 8),  
18  ('X4579462R', 'Bases de Datos', '25/26', 6, 9),  
19  ('X4579462R', 'Sistemas Informáticos', '25/26', 6, 7);
```

Creación de componente

He creado el proyecto AD07_Alumno donde he creado 2 clases AlumnoBean.java y MatriculaBean.java

En la clase **AlumnoBean.java** empiezo indicando que implemente Serializable para que los componentes puedan persistir. También creo la variable propertySupport de tipo PropertyChangeSupport para controlar y lanzar los eventos cuando se cargan todas las matrículas o solo las matrículas de un alumno.

Y dentro del Constructor de la clase inicializo el soporte de eventos y recargo las filas.

```
14
15 public class AlumnoBean implements Serializable{
16
17     // Propiedades del Bean
18     private PropertyChangeSupport propertySupport;
19
20     public AlumnoBean(){
21         // Inicializo soporte de eventos
22         propertySupport = new PropertyChangeSupport(this);
23
24         // Cargo los datos desde la base de datos
25         recargarFilas();
26     }
```

Eugen Moga

También creo los getter y setter de todas las propiedades de la clase AlumnoBean.

Dentro de la clase AlumnoBean.java creo una clase privada Alumno como clase auxiliar para poder crear un vector privado de alumnos. Y un constructor vacío.

```
70 // Clase auxiliar para crear
71 private class Alumno {
72
73     String DNI;
74     String nombre;
75     String apellidos;
76     String direccion;
77     Date fechaNac;
78
79     // Constructor vacío
80     public Alumno(){
81     }
```

Tambien creo el constructor con parámetros

```
83 // Constructor con parametros
84 public Alumno(String nDNI, String nNombre, String nApellidos, String nDireccion, Date nFechaNac){
85     this.DNI = nDNI;
86     this.nombre = nNombre;
87     this.apellidos = nApellidos;
88     this.direccion = nDireccion;
89     this.fechaNac = nFechaNac;
90 }
91
```

Utilizo el vector para guardar los Alumnos, des este modo cargo la información en el vector para no necesitar estar conectado continuamente a la base de datos.

```
93 // Utilizo un vector auxiliar para carga
94 // sin necesidad de estar conectados con
95 private Vector Alumnos = new Vector();
```

Creo el método recargarFilas para actualizar el contenido de la tabla en el vector de alumnos.

Dentro de un bloque try-catch le paso la conexión con este enlace:
"jdbc:mysql://127.0.0.1:3306/RRHH?useSSL=false", "accesodatos",
"A1b2c3d4."

Aquí vuelvo a mencionar que estoy usando la base de datos de la tarea del tema 3 donde el nombre de la base de datos es RRHH el usuario es accesodatos y la contraseña A1b2c3d4.

```
97 // Actualiza el contenido de la tabla en el vector de alumnos
98 private void recargarFilas(){
99     try {
100         Connection con = DriverManager.getConnection("jdbc:mysql://127.0.0.1:3306/RRHH?useSSL=false", "accesodatos", "A1b2c3d4.");
101         Statement s = con.createStatement();
102         ResultSet rs = s.executeQuery("select * from alumnos");
103         while (rs.next()){
104             Alumno a = new Alumno(rs.getString("DNI"),
105                                   rs.getString("Nombre"),
106                                   rs.getString("Apellidos"),
107                                   rs.getString("Direccion"),
108                                   rs.getDate("FechaNac"));
109             Alumnos.add(a);
110         }
111         Alumno a = new Alumno();
112         a = (Alumno) Alumnos.elementAt(1);
113         this.DNI = a.DNI;
114         this.nombre = a.nombre;
115         this.apellidos = a.apellidos;
116         this.direccion = a.direccion;
117         this.fechaNac = a.fechaNac;
118         rs.close();
119         con.close();
120     } catch (SQLException ex){
121         this.DNI = "";
122         this.nombre = "";
123         this.apellidos = "";
124         this.direccion = "";
125         Logger.getLogger(AlumnoBean.class.getName()).log(Level.SEVERE, null, ex);
126     }
127 }
```

Eugen Moga

Utilizo el método seleccionarFila para indicar el numero de la fila a cargar en las propiedades del componente.

```
// Numero de la fila a cargar en las propiedades del componente
public void seleccionarFila(int i){
    if (i <= Alumnos.size()){
        Alumno a = new Alumno();
        a = (Alumno) Alumnos.elementAt(i);
        this.DNI = a.DNI;
        this.nombre = a.nombre;
        this.apellidos = a.apellidos;
        this.direccion = a.direccion;
        this.fechaNac = a.fechaNac;
    }else {
        this.DNI = "";
        this.nombre = "";
        this.apellidos = "";
        this.direccion = "";
    }
}
```

Creo el método seleccionarDNI para indicar el dni a buscar en las propiedades del componente

```
148 // DNI a buscar y se cargar en las propiedades del compon
149 public void seleccionarDNI(String nDNI){
150     Alumno a = new Alumno();
151     int i=0;
152
153     this.DNI = "";
154     this.nombre = "";
155     this.apellidos = "";
156     this.direccion = "";
157     while (this.DNI.equals("") && i <= Alumnos.size()){
158         a = (Alumno)Alumnos.elementAt(i);
159         if (a.DNI.equals(nDNI)){
160             this.DNI = nDNI;
161             this.nombre = a.nombre;
162             this.apellidos = a.apellidos;
163             this.direccion = a.direccion;
164             this.fechaNac = a.fechaNac;
165         }
166         i++;
167     }
168 }
```


En la clase **MatriculaBean.java** empiezo creando una variable `propertySupport` de tipo `PropertyChangeSupport` para las propiedades del bean.

Agrego un constructor sin parámetros donde inicializo el soporte de eventos `propertySupport`.

```
22 public class MatriculaBean {
23
24     // Propiedades del bean
25     private PropertyChangeSupport propertySupport;
26
27     /**
28      * Método constructor por defecto del componente
29      */
30     public MatriculaBean() {
31         // Inicializo soporte de eventos
32         propertySupport = new PropertyChangeSupport(this);
33     }
34
```

También creo los `getter` y `setter` de todas las propiedades de la clase `MatriculaBean.java`

```
35     private String dni;
36
37     /**
38      * Get the value of dni
39      *
40      * @return the value of dni
41      */
42     public String getDni() {
43         return dni;
44     }
45
46     /**
47      * Set the value of dni
48      *
49      * @param dni new value of dni
50      */
51     public void setDni(String dni) {
52         this.dni = dni;
53     }

```

Dentro de la clase MatriculaBean.java creo una clase privada Matricula con todas las propiedades.

```
138 private class Matricula {
139
140     private final String dni;
141     private final String asignatura;
142     private final String cursoEscolar;
143     private final int credits;
144     private final int calificacion;
145 }
```

Creo el constructor por defecto e inicializo todos los valores vacíos.

```
149 public Matricula() {
150     this.dni = "";
151     this.asignatura = "";
152     this.cursoEscolar = "";
153     this.credits = -1;
154     this.calificacion = -1;
155 }
```

Tambien creo el constructor parametrizado

```
166 public Matricula(String nDNI, String nAsignatura, String nCursoEscolar, int nCredits, int nCalificacion)
167 {
168     this.dni = nDNI;
169     this.asignatura = nAsignatura;
170     this.cursoEscolar = nCursoEscolar;
171     this.credits = nCredits;
172     this.calificacion = nCalificacion;
173 }
```

Creo el vector de la clase matricula para cargar la información de la base de datos

```
175 /**
176  * Vector auxiliar para cargar la información de la base de datos.
177  */
178 private Vector<Matricula> matriculas = new Vector<>();
179
```

Creo el método seleccionarRegistro donde le paso un numero entero que hace referencia al numero de fila y selecciono todas las propiedades de ese componente matricula

```
195 public void seleccionarRegistro(int i) {
196     Matricula matricula;
197
198     if (i < matriculas.size()) { //Comprueba que exist
199         matricula = matriculas.get(i);
200         this.dni = matricula.dni;
201         this.asignatura = matricula.asignatura;
202         this.cursoEscolar = matricula.cursoEscolar;
203         this.creditos = matricula.creditos;
204         this.calificacion = matricula.calificacion;
205     }
206 }
```

Creo el método recargarDNI donde le paso el DNI en formato String. Creo las variables de tipo Connection, PreparedStatement y ResultSet para tratar la conexión y el tratamiento de datos de la base de datos. Y limpio el vector antes de empezar.

```
216 public void recargarDNI(String nDni) {
217     Connection con;
218     PreparedStatement pstmt;
219     ResultSet rs;
220     matriculas.clear(); // Limpia el vector
```

Dentro del bloque try-catch le paso la conexión a la base de datos con nombre de la base de datos, usuario y contraseña. Seguido hago una comprobación si el DNI este vacío genero el evento y después le paso la consulta SQL para seleccionar todas las matrículas.

```
221 try {
222     con = DriverManager.getConnection("jdbc:mysql://127.0.0.1:3306/RRR");
223     if (nDni.isBlank() || nDni.isEmpty()) { //Si no se
224         receptor.capturarRecarga(new recargaEvent(this)); //Genera
        pstmt = con.prepareStatement("select * from matriculas");
```

Con else compruebo si ha introducido el DNI y genero el evento para el DNI. Después la paso la consulta SQL para que seleccione todas las matriculas que coinciden con el DNI.

Ejecuto la consulta

```
226 } else { //Si se indica un número de dni
227     receptor.capturarRecargaDNI(new recargaDniEvent(this)); //Genera un ev
228     pstmt = con.prepareStatement("select * from matriculas where dni = ?");
229     pstmt.setString(1, nDni);
230 }
231 rs = pstmt.executeQuery();
```

Dentro de un bucle while reitero sobre los resultados y guardo todo en el vector matricula. Cierro las conexiones. Y dentro del catch trato la excepción SQL si se genera algún problema.

```
232 while (rs.next()) { //Copia los datos de cada matricula
233     Matricula matricula = new MatriculaBean.Matricula(
234         rs.getString("DNI"),
235         rs.getString("Asignatura"),
236         rs.getString("CursoEscolar"),
237         rs.getInt("Creditos"),
238         rs.getInt("Calificacion"));
239     matriculas.add(matricula); //Añade la matrícula al vector
240 }
241 rs.close();
242 pstmt.close();
243 con.close();
244 } catch (SQLException ex) {
245     Logger.getLogger(MatriculaBean.class.getName()).log(Level.SEVERE, null, ex);
246 }
```

Creo el método addMatricula para agregar un matricula. Dentro de un bloque try catch le paso la conexión captúralo el evento y luego la paso el INSERT SQL con los datos. Para finalizar en el catch trato la excepción SQL.

```

253 public void addMatricula() {
254     Connection con;
255     PreparedStatement pstmt;
256     try {
257         con = DriverManager.getConnection("jdbc:mysql://127.0.0.1:3306/RRHH?useSSL=false", "acc
258         receptor.capturarBDModificada(new bdModificadaEvent(this)); //Genera un evento
259         pstmt = con.prepareStatement("insert into matriculas values (?, ?, ?, ?, ?)");
260         pstmt.setString(1, this.dni);
261         pstmt.setString(2, this.asignatura);
262         pstmt.setString(3, this.cursoEscolar);
263         pstmt.setInt(4, (int) this.creditos);
264         pstmt.setInt(5, (int) this.calificacion);
265         pstmt.executeUpdate(); //Añade la matrícula a la base de datos
266     } catch (SQLException ex) {
267         Logger.getLogger(MatriculaBean.class.getName()).log(Level.SEVERE, null, ex);
268     }
269 }

```

Para tratar el tema de los eventos creo por cada evento creo una clase y le indico que hereda de EventoObject y en la clase le pongo el constructor indicando Object source que es el objeto que genero el evento.

```

277 public class bdModificadaEvent extends java.util.EventObject {
278     public bdModificadaEvent(Object source) {
279         super(source);
280     }
281 }

```

```

287 public class recargaEvent extends java.util.EventObject {
288     public recargaEvent(Object source) {
289         super(source);
290     }
291 }

```

```

297 public class recargaDniEvent extends java.util.EventObject {
298     public recargaDniEvent(Object source) {
299         super(source);
300     }
301 }

```

Creo la variable privada receptor de tipo interfaceListener. Y defino una interfaz publica llamada interfaceListener que le indico que extiende de EventListener esto es para marcar que esta interfaz es un escuchador de eventos.

Luego creo los métodos públicos para cada evento.

```
306     private interfaceListener receptor;
307
308     public interface interfaceListener extends EventListener {
309
310         public void capturarBDModificada(bdModificadaEvent ev);
311
312         public void capturarRecarga(recargaEvent ev);
313
314         public void capturarRecargaDNI(recargaDniEvent ev);
315
316     }
```

Ahora creo los métodos para registrar los listener para guardar una referencia al objeto que quiere escuchar eventos

```
318     /**
319      * Añade un listener
320      *
321      * @param receptor
322      */
323     public void addInterfaceListener(interfaceListener receptor) {
324         this.receptor = receptor;
325     }
```

El método para eliminar el listener lo que hace es eliminar el listener registrado y a partir de ese momento no se enviaran mas eventos.

```
327     /**
328      * Elimina un listener
329      *
330      * @param receptor
331      */
332     public void removeInterfaceListener(interfaceListener receptor) {
333         this.receptor = null;
334     }
```

Este método añade un listener para notificar cualquier cambio en alguna propiedad del bean.

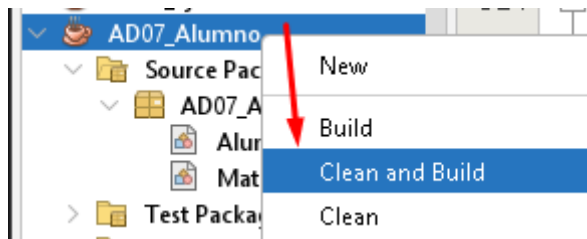
```
341 public void addPropertyChangeListener(PropertyChangeListener listener) {  
342     propertySupport.addPropertyChangeListener(listener);  
343 }
```

Este método elimina el listener previamente registrado.

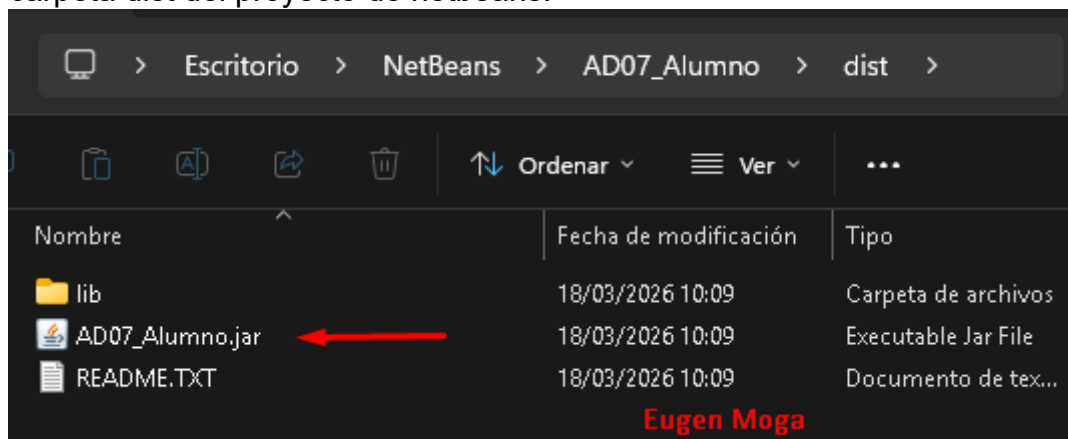
```
345 public void removePropertyChangeListener(PropertyChangeListener listener) {  
346     propertySupport.removePropertyChangeListener(listener);  
347 }  
348 }
```

Ahora que tengo que tengo la clase AlumnoBean y MatriculaBean terminadas compilo el proyecto para generar el archivo .jar que voy a necesitar en el proyecto prueba para importar y poder usar todos los métodos necesarios que tengo en la clase MatriculaBean.

Le pulso clic derecho en el proyecto y selecciono Clean and Build.



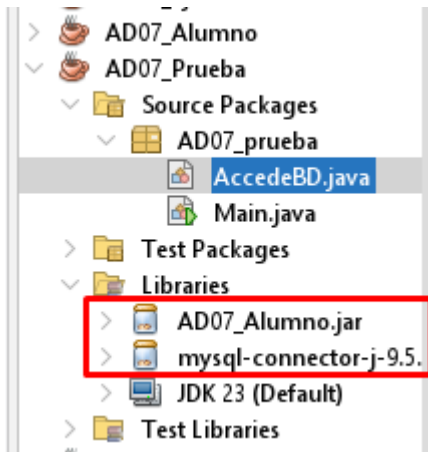
Esto me compila el proyecto y me genera el archivo AD07_Alumno.jar en la carpeta dist del proyecto de netbeans.



Proyecto AD07_Prueba

Creo el proyecto nuevo para hacer las pruebas y dentro una clase AccedeBD.java

Para poder importar y usar los métodos del AD07_Alumno tengo que agregar en la librería del nuevo proyecto el archivo AD07_Alumno.jar creado anteriormente. Y al igual que en el caso anterior tambien agrego el conector mysql para establecer la conexión con la base de datos.



Importo el JavaBean que gestiona las matriculas de los alumnos.

```
5
6 import AD07_Alumno.MatriculaBean;
7
```

En la clase AccedeBD implemento la interfaz de eventos para reaccionar a cambios que se generan en la BD. Creo la instancia de MatriculaBean y dentro del constructor creo el objeto matriculas y registro el listener para recibir eventos.

```
12 public class AccedeBD implements MatriculaBean.interfaceListener{
13
14     // Creo una instancia del bean que gestiona las matriculas
15     MatriculaBean matriculas;
16
17     public AccedeBD() {
18         matriculas = new MatriculaBean();
19         matriculas.addInterfaceListener(this);
20     }
21 }
```


Creo el método listadoMatriculas donde cargo todas las matriculas para ello llamo al método recargarDNI y el DNI lo dejo en blanco para listar todas las matrículas.
Con un bucle for recorro todas las matrículas y y muestro los datos en pantalla.

```
22 // Método que lista todas las matriculas de la base de datos
23 public void listadoMatriculas(){
24     matriculas.recargarDNI("");
25     for (int i = 0; i < matriculas.size(); i++){
26         matriculas.seleccionarRegistro(i);
27         System.out.println("Matricula: " + (i + 1));
28         System.out.println("\tDNI: " + matriculas.getDni());
29         System.out.println("\tAsignatura: " + matriculas.getAsignatura());
30         System.out.println("\tCurso Escolar: " + matriculas.getCursoEscolar());
31         System.out.println("\tCreditos: " + matriculas.getCreditos());
32         System.out.println("\tCalificación: " + matriculas.getCalificacion() + "\n");
33     }
34 }
```

Creo el método listadoMatriculasDNI este método es idéntico al anterior solo que aquí al llamar al método recargarDNI le indico un DNI a buscar

```
36 // Método que lista las matrículas de un DNI especificado
37 public void listadoMatriculasDNI(){
38     matriculas.recargarDNI("X4579462R");
39     for (int i = 0; i < matriculas.size(); i++){
40         matriculas.seleccionarRegistro(i);
41         System.out.println("Matricula: " + (i + 1));
42         System.out.println("\tDNI: " + matriculas.getDni());
43         System.out.println("\tAsignatura: " + matriculas.getAsignatura());
44         System.out.println("\tCurso Escolar: " + matriculas.getCursoEscolar());
45         System.out.println("\tCreditos: " + matriculas.getCreditos());
46         System.out.println("\tCalificación: " + matriculas.getCalificacion() + "\n");
47     }
48 }
```

Creo el método `anadeMatricula` y con los setter le indico todos los parámetros de la matricula que voy a añadir. Después muestro los datos en pantalla y para finalizar llamo al método `addMatricula` para agregar la matricula a la base de datos.

```
50 // Método para añadir una matrícula nueva
51 public void anadeMatricula(){
52     matriculas.setDni("X4579462R");
53     matriculas.setAsignatura("Desarrollo Interfaces");
54     matriculas.setCursoEscolar("25/26");
55     matriculas.setCreditos(6);
56     matriculas.setCalificacion(9);
57     System.out.println("Matricula nueva: ");
58     System.out.println("\tDNI: " + matriculas.getDni());
59     System.out.println("\tAsignatura: " + matriculas.getAsignatura());
60     System.out.println("\tCurso Escolar: " + matriculas.getCursoEscolar());
61     System.out.println("\tCreditos: " + matriculas.getCreditos());
62     System.out.println("\tCalificación: " + matriculas.getCalificacion() + "\n");
63     matriculas.addMatricula();
64 }
```

Al finalizar sobrescribo los eventos que se generan cuando se modifica la base de datos para mostrar los mensajes de forma clara.

```
67 @Override
68 public void capturarBDModificada(MatriculaBean.bdModificadaEvent me) {
69     System.out.println("\nSe ha añadido una matricula a la base de datos\n");
70 }
71
72 @Override
73 public void capturarRecarga(MatriculaBean.recargaEvent r) {
74     System.out.println("\nSe han recargado todas las matriculas\n");
75 }
76
77 @Override
78 public void capturarRecargaDNI(MatriculaBean.recargaDniEvent de) {
79     System.out.println("\nSe han recargado las matriculas de un dni\n");
80 }
81 }
```

Creo otra clase Main que contendrá el método main y será donde probaré todo el funcionamiento.

Primero creo un objeto de la clase AccedeBD y paso a llamar a los métodos correspondientes. Primero llamo al método para listar todas las matrículas, después listo las matrículas de un DNI, después añado una matrícula y para finalizar vuelvo a listar las matrículas de un DNI para reflejar los cambios.

```
19  ...  
20  public static void main(String[] args) {  
21      // Creo un objeto de la clase AccedeBD  
22      AccedeBD gestion = new AccedeBD();  
23  
24      gestion.listadoMatriculas();  
25      gestion.listadoMatriculasDNI();  
26      gestion.anadeMatricula();  
27      gestion.listadoMatriculasDNI();  
28  }  
29  }
```

Voy a probar el funcionamiento ejecutando el proyecto.
Puedo ver que me lanza el mensaje se han recargado todas las matrículas porque ha capturado el evento.

```
Output - AD07_Prueba (run)

run:
Picked up JAVA_TOOL_OPTIONS: -Dfile.encoding=UTF

Se han recargado todas las matriculas

Matricula: 1
  DNI: 12345678A
  Asignatura: Bases de Datos
  Curso Escolar: 23/24
  Creditos: 6
  Calificación: 8

Matricula: 2
  DNI: 12345678A
  Asignatura: Lenguajes de Marcas
  Curso Escolar: 23/24
  Creditos: 4
  Calificación: 9

Matricula: 3
  DNI: 12345678A
  Asignatura: Programación
  Curso Escolar: 23/24
  Creditos: 8
  Calificación: 7

Matricula: 4
  DNI: 14785236D
  Asignatura: Bases de Datos
  Curso Escolar: 23/24
  Creditos: 6
  Calificación: 5

Matricula: 5
  DNI: 14785236D
  Asignatura: Programación
  Curso Escolar: 23/24
  Creditos: 8
  Calificación: 6

Matricula: 6
  DNI: 23456789B
  Asignatura: Entornos de Desarrollo
  Curso Escolar: 24/25
  Creditos: 3
  Calificación: 8

Matricula: 7
  DNI: 23456789B
  Asignatura: Programación
```

Eugen Moga

Output Inspector

Después puedo ver como me captura el evento y me muestra el mensaje que se han cargado las matrículas de un DNI

```
Se han recargado las matriculas de un dni ←  
  
Matricula: 1  
  DNI: X4579462R  
  Asignatura: Bases de Datos  
  Curso Escolar: 25/26  
  Creditos: 6  
  Calificación: 9  
  
Matricula: 2  
  DNI: X4579462R  
  Asignatura: Programación  
  Curso Escolar: 25/26  
  Creditos: 8  
  Calificación: 8  
  
Matricula: 3  
  DNI: X4579462R  
  Asignatura: Sistemas Informáticos  
  Curso Escolar: 25/26  
  Creditos: 6  
  Calificación: 7
```

Después me muestra la matricula nueva y los datos añadidos y al finalizar me captura el evento y me muestra el mensaje que se ha añadido una matricula a la base de datos

```
Matricula nueva: ←  
  DNI: X4579462R  
  Asignatura: Desarrollo Interfaces  
  Curso Escolar: 25/26  
  Creditos: 6  
  Calificación: 9  
  
Se ha añadido una matricula a la base de datos ←
```

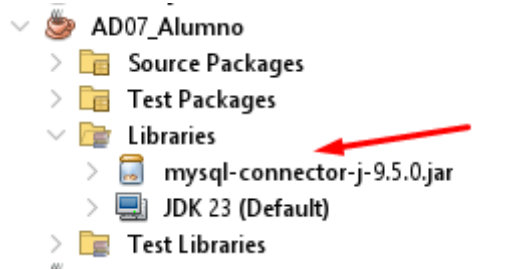
Este evento me lo captura después de mostrar los datos en pantalla porque en el método anadeMatricula primero muestro los datos en pantalla y después modifico la base de datos.

Para finalizar vuelvo a listar las matriculas de un DNI y puedo ver como me lista la matricula añadida anteriormente

```
Se han recargado las matriculas de un dni   
  
Matricula: 1  
  DNI: X4579462R  
  Asignatura: Bases de Datos  
  Curso Escolar: 25/26  
  Creditos: 6  
  Calificación: 9  
  
Matricula: 2  
  DNI: X4579462R  
  Asignatura: Desarrollo Interfaces   
  Curso Escolar: 25/26  
  Creditos: 6  
  Calificación: 9  
  
Matricula: 3  
  DNI: X4579462R  
  Asignatura: Programación  
  Curso Escolar: 25/26  
  Creditos: 8  
  Calificación: 8  
  
Matricula: 4  
  DNI: X4579462R  
  Asignatura: Sistemas Informáticos  
  Curso Escolar: 25/26  
  Creditos: 6  
  Calificación: 7  
  
BUILD SUCCESSFUL (total time: 1 second)
```

Aparece en segunda posición porque lo esta ordenando alfabéticamente.

Importante para que esto funcione en el proyecto AD07_Alumno agregar el conector mysql en la librería del proyecto



También hay que tener en cuenta el nombre de la base de datos con su usuario y contraseña.

Yo he usado la base de datos RRHH del tema 3 con su usuario: accesodatos y contraseña: A1b2c3d4.